

Org structure, mapping, job posting, and the user

This document describes how the live WillsOne code stores company catalogs, how those catalogs are mapped into allowed combinations, how a job posting is created from that tree, and how a hired person becomes a users row with the same placement.

Source of truth: the application code and SQL in this repo — not a separate design spec. If the product and this PDF disagree, the code wins.

Contents

1. The one chain that matters
2. Catalogs — Organizational Structure lists
3. Mapping — which combinations are valid
4. Create job posting
5. Apply, onboard, invite — posting becomes a user
6. What lives on the employee after hire
7. Downstream: appraisal and skill log templates
8. Facts that must not be mixed

1. The one chain that matters

Four layers share the same foreign keys. A Site / Business unit / Department / Section / Position / Grade level combination is defined in catalogs, restricted by mapping, stamped onto a job posting, then copied onto the hired employee. After hire, Access Control can still edit that placement on the user - the posting is the source at invite time, not a live join forever.

1 Catalogs - org_custom_list_types + one physical table per list (sites, business_units, ...)



2 Mapping - org_mapping_levels + org_map_* tables (valid child IDs under a parent path)



3 Job posting - job_postings holds one real FK column per list (site_id, position_id, ...)



4 Application - job_applications.job_posting_id points at that posting



5 User - users gets the same FKs at invite, plus supervisor_id and application_id

Where this is configured in the product

System Definitions -> Organizational structure (lists). System Definitions -> Org structure mapping set up (allowed tree).

System Definitions -> Create job posting (the vacancy). Recruitment -> apply / interview / onboarding -> Invite to WillsOne (the user).

2. Catalogs - Organizational Structure lists

Every list - the original five plus anything added from Set up - is a row in org_custom_list_types. Each row names its own physical Postgres table. The five original catalogs were not renamed when they were folded into this registry; sites, business_units, departments, sections, and grade_levels still exist as those tables.

Built-in lists the rest of hiring depends on

List (table_name)	Typical job_postings column	Role in hiring
Sites (sites)	site_id	Root of the org path. Label + region become the posting location.
Business units (business_units)	business_unit_id	Second step of the required cascade.
Departments (departments)	department_id	Third step of the required cascade.
Sections (sections)	section_id	Fourth step of the required cascade.
Position (custom_position)	position_id	Fifth step. The Position label becomes the posting title and slug.
Grade levels (grade_levels)	grade_level_id	Required on the posting. Copied to users.grade_level_id. Display code comes from grade_levels.code via join - not a free-text users.grade_level column.
Salary (custom_salary)	salary_id	Required on the posting (bands). Offer terms read salary from the posting, not from a separate pay table.
Employment type (custom_employment_type)	employment_type_id (or similar)	Label is copied into job_postings.employment_type text.

Age (custom_age)	age_id, plus age_min_id / age_max_id	Digits-mode range. AI screening reads the posting's age band.
User role (custom table, by label)	user_role_id	Copied to users.user_role_id. This is who may act as a supervisor - not users.supervisor_id.

Admins can add more lists from Organizational structure -> Add new list. Creating a list creates a physical table and a matching foreign-key column on job_postings (add_job_posting_org_column). Digits-mode numeric lists also get min/max columns so a posting can store a range of catalog rows, not raw numbers. Disabled lists (is_active = false) are hidden from Create job posting; existing rows and references stay.

Grade levels catalog extra columns

grade_levels also carries rank, role_kind (ranked | consultant), age_min, and age_max. fetchGradeLevelsConfig reads this table - there is no hardcoded L1-L7 fallback in that path. users.grade_level (text) is retired; users.grade_level_id is the FK.

3. Mapping - which combinations are valid

Catalogs are flat. Mapping is the tree that says "this Business unit exists under this Site," and so on. Without mapping, every Site could pair with every Position. Mapping is what Create job posting, Access Control org placement, Appraisal scope, and Skill log scope all consult - through two read shapes described below.

Registry: org_mapping_levels

One row per participating list. parent_level_id is the parent in the tree. Site is the root (no parent). The mapping-setup page auto-seeds this required chain if it is missing, and those five levels cannot be removed, because Create job posting's cascade depends on them:

```
sites -> business_units -> departments -> sections -> custom_position
```

Grade level is not part of that required five. It is still a catalog, still required on a posting, and it has its own mapping table (org_map_grade_levels) used by Appraisal / Skill log templates so a grade is only offered under a full org path. Admins may add extra levels (for example User role under Position).

Physical mapping tables (org_map_*)

Every non-root level has a real table named org_map_<list table>, with one real FK column per ancestor plus itself. A unique constraint on that column set means each combination is stored once.

Table	FK columns (root -> self)
org_map_business_units	site_id, business_unit_id
org_map_departments	site_id, business_unit_id, department_id
org_map_sections	site_id, business_unit_id, department_id, section_id
org_map_custom_position	site_id, business_unit_id, department_id, section_id, position_id
org_map_grade_levels	site_id, business_unit_id, department_id, section_id, position_id, grade_level_id

Site has no org_map_sites table. There is nothing to constrain a root against. Checked Sites (rare root checkboxes) still live in the older shared table org_mapping_nodes.

How the mapping UI writes, and how screens read

Org structure mapping set up talks to GET/POST/DELETE /api/organizational-structure/mapping-nodes. That API is a translator. Root rows still go to org_mapping_nodes. Every other level is inserted into the matching org_map_* table with the full ancestor chain filled in. The JSON the UI sees is always a flat node: { id, level_id, item_id, parent_node_id }, with id

shaped as `level_id::rawId` so the page never has to know which storage was used.

Two consumers then read that same data differently:

Consumer	API / helper	Empty mapping behaviour
Create job posting cascade, and Access Control org placement	GET mapping-nodes, then walk <code>parent_node_id</code>	Fail open. If a level has no nodes yet, the dropdown shows the whole catalog so hiring is not blocked while mapping is rolled out one level at a time.
Appraisal grade templates and Skill log templates	GET <code>/organizational-structure/org-maps + itemsForOrgMapField</code>	Fail closed. Child lists only include IDs that exist in <code>org_map_*</code> under the selected parents. No parent selected -> empty child list.

Do not use `org_mapping_levels` / `org_mapping_nodes` as the Appraisal or Skill log picker source

Those screens load the real `org_map_*` rows. Labels still come from the catalog tables. Mapping rows only decide which IDs are valid under the current pick.

4. Create job posting

The page lives at System Definitions -> Create job posting (not on the Recruitment careers list - that list is for operating published roles). Creating a posting is a two-step save: Details, then Interview setup for the same posting id.

Always-on cascade (five fields)

Site, Business unit, Department, Section, and Position are always shown and always required. Changing a parent clears downstream picks. Options for each child are filtered by the mapping-nodes tree described above. Title is not typed: `resolveTitleFromPosition` reads `custom_position.label`. The slug is a unique snake_case of that title (`generateUniquePostingSlug`).

Also required, but not in that cascade

- Grade level - every posting needs a grade so the hired user and offer/appraisal matching have `grade_level_id`.
- Salary - Offer Terms and onboarding HR fields read salary only from the linked posting.

On the form, every other active list (Age, Employment type, User role, future custom lists) is opt-in per posting. The order those extras were added is stored on `job_postings.optional_org_field_order` so they do not jump around by list `sort_order`. Numeric Age can be one value or min/max, both still FKs into `custom_age`.

Server vs form on "all fields required"

The Details form requires the cascade plus Grade and Salary, and any extras the HR added. `POST /api/careers/postings` (new posting, not Republish) runs `findMissingOrgFields` against every active org list. Republish of a closed posting carries org fields forward and does not re-apply that full-list rule, so a legacy posting can be reopened without backfilling missing catalogs.

Derived posting fields (not separate catalogs)

Stored on <code>job_postings</code>	Comes from
<code>title</code> , <code>slug</code> , <code>job_title_key</code>	Position label
<code>location</code>	Site label + Site region (Ghana region list)
<code>employment_type</code> (text)	Employment type item label
<code>summary</code>	Truncated description preview

closes_at / status	Ghana-local closing datetime; published vs closed
role_scope, key_responsibilities, ...	JD sections; may be extracted from an uploaded JD
interview_setup, panel, duration	Interview step after Save

A published posting appears on the public careers page. Closing hides it from apply. Archive hides it from the main Create job posting table. Republish creates a new row and sets superseded_by on the old one.

5. Apply, onboard, invite - posting becomes a user

The posting does not create a user. The hire pipeline does. The link that carries org placement is always job_posting_id on the application, then application_id on the user.

Public apply -> job_applications row with job_posting_id, reference number, form JSON



Interview / shortlist -> still keyed by that application (and therefore that posting)



Onboarding submission -> candidate form + HR Section O (employee id, company email, supervisor)



Senior HR / authorised consultant approves -> POST /api/careers/onboarding/finish-hr



resolveEmployeeOrgPlacementFromPosting(job_posting_id) copies the six path FKs + user_role_id



invitePlatformEmployee inserts users with those FKs, supervisor_id, application_id, auth invite

resolveEmployeeOrgPlacementFromPosting selects site_id, business_unit_id, department_id, section_id, position_id, grade_level_id, and user_role_id from job_postings. Nulls mean a legacy posting or a missing list - they are not invented. Existing employees hired before org columns existed can be backfilled with docs/access-control/backfill-employee-org-placement.sql (only fills still-null columns from the linked posting).

The invite also writes job_position as text (from the application / HR position title) for display, and may resolve a leftover text grade hint to grade_levels.id if the posting did not already supply grade_level_id. Supervisor is not on the posting. It is chosen in onboarding HR review and stored as users.supervisor_id, after canAssignAsSupervisor checks the target person's User role.

6. What lives on the employee after hire

users is the employee record. Org placement columns are the same physical FKs as the posting (ON DELETE SET NULL - deleting a Site never deletes a person).

users column	Meaning	Copied from posting?
site_id ... position_id	Org path	Yes, at invite
grade_level_id	Grade catalog row. Display via grade_levels(code)	Yes, at invite
user_role_id	Whether they may act as supervisor / which permission group	Yes, if the posting had User role
supervisor_id	Reporting line: whose team they are on	No - set in onboarding / Access Control
application_id	The job_applications row they were hired from	N/A - set at invite

job_position	Free-text title snapshot	From application / HR title, not the Position FK itself
employment_status	e.g. probation on first invite	No

After invite, Access Control -> that user -> Org placement can change the same seven FKs. Dropdowns there reuse the mapping tree (fail-open like Create job posting). Appraisal and skill log then read the user's current placement, not the original posting. If HR later moves someone, their next appraisal looks up templates for the new combination.

Two different supervisor_id columns

users.supervisor_id is the reporting line (who they report to). appraisals.supervisor_id and skill_logs.supervisor_id are who filled that record. Do not treat a filled appraisal's supervisor_id as the employee's manager, and do not drop users.supervisor_id - assignment still scopes "people I supervise." User role opens the supervisor tab even if that list is empty.

7. Downstream: appraisal and skill log templates

Once the user has a complete org placement, live Appraisal and Skill log forms look up a template with all six FKs equal - no partial match:

```
site_id + business_unit_id + department_id + section_id + position_id + grade_level_id
```

Those templates are authored under System Definitions (Appraisal scope / Skill log scope) using the fail-closed org_map_* pickers. Incomplete user placement, or a combination with no template, shows the same empty states as "place this person first" / "no template for this path." That is why mapping and posting and hire all have to carry the same IDs: the later modules never re-derive org from a job title string.

8. Facts that must not be mixed

Fact	Source	Not this
What lists exist	org_custom_list_types + physical catalog tables	Hardcoded L1-L7 or a retired job-title-options list
Which combinations are allowed	org_map_* (and root checkboxes in org_mapping_nodes)	A free join of every catalog row to every other catalog row
What this vacancy is	job_postings FKs + Position-derived title	A typed job title disconnected from Position
What this employee is, org-wise	users.site_id ... grade_level_id (copied at hire, editable later)	Re-reading the posting on every appraisal
May they act as a supervisor?	users.user_role_id -> User role catalog (Supervisory / Executive / HR / Super Admin)	Whether anyone currently has supervisor_id = them
Who reports to them?	users.supervisor_id of other people	appraisals.supervisor_id / skill_logs.supervisor_id

Primary code and SQL to open

- src/lib/organizationalStructureCustomLists.ts - list registry types.
- src/lib/organizationalStructureMapping.ts - fail-closed org_map_* helper.
- src/app/api/organizational-structure/mapping-nodes/route.ts - mapping UI translator.
- src/lib/careers/jobPostingOrgFields.ts - posting FK whitelist, title from Position.
- src/app/(dashboard)/dashboard/system-definitions/create-job-posting/page.tsx - create UI.
- src/lib/careers/resolveEmployeeOrgPlacement.ts - posting -> user copy.

- `src/app/api/careers/onboarding/finish-hr/route.ts` - hire invite.
- `docs/access-control/users-org-placement.sql` and `users-org-placement-user-role.sql`.
- `docs/organizational-structure/org-structure-mapping-real-tables.sql` and `job-postings-org-fields.sql`.

How to use this PDF

Walk a real posting: pick a mapped Site -> BU -> Department -> Section -> Position -> Grade, save, apply as a candidate, finish onboarding, then open that user in Access Control. Confirm the seven placement FKs match the posting (except `supervisor_id`, which you set in HR review). Then open Appraisal for that employee and confirm the template match uses those same six IDs.